Reproducing CARE

A complete working guide: from the paper to a result you can explain in public

Paper. “CARE: Confidence-Aware Ratio Estimation for Medical Biomarkers”, Li, Popordanoska, Tiulpin, Gruber, Maes, Blaschko, MIDL 2026 (PMLR 315, pp. 172-197). Earlier preprint: arXiv 2505.19585, “Beyond Segmentation: Confidence-Aware and Debiased Estimation of Ratio-based Biomarkers”.

What we reproduce. The construction of confidence intervals for the necrosis-to-tumour ratio (NTR) from the output of a segmentation network, and the check of how often those intervals contain the true value (coverage). Data: Medical Segmentation Decathlon, Task01 (brain tumour MRI). Model: nnU-Net 3D.

Goal of this document. To be a single reference that tells you, at every moment, what you are doing, where you are doing it (VS Code on your laptop, or a Kaggle notebook), why it is done there, and how. After reading it you should be able to present the full pipeline without notes.

Part	Content	When to read
1	The big picture and the 9 steps	First, once
2	The paper explained (ideas, maths, results)	First, once; re-read before step 5
3	Scope: what we reproduce, what we skip	First, once
4	Where each task runs and why (laptop vs Kaggle)	Before step 1
5	The pipeline schema (landscape page)	Keep it open all along
6	Step-by-step instructions (steps 1-9)	During the work
7	Inside stats.npz and how CARE uses it	Steps 5, 6 and 8
8	Repository layout, time and cost, risks, deviation log	Throughout
9	Presentation kit: story, questions and answers, glossary	Before presenting
A	Checklists: facts to verify, open decisions	Early, then at the end

Honesty rule. Facts about the paper in this guide come from reading the arXiv v2 text through a summarizing tool, not from a line-by-line reading of the PDF. Anything marked **[VERIFY]** must be checked against the PDF before you rely on it or quote it. Time, price and platform-limit figures are estimates from memory; measure or check them on your accounts. Version 1, 31 October 2026.

Part 1. The big picture

The idea in one paragraph (say it aloud until it is natural)

Doctors often use a ratio computed from an AI segmentation, for example the share of a brain tumour that is necrotic. The AI gives one number, with no indication of how much to trust it, and if that number is close to a clinical threshold, a small error can change a decision. CARE adds the missing piece: a **confidence interval** around the ratio, built *after* the network is trained, from two ingredients: the random error of a ratio of two sums, and the miscalibration of the network measured on a held-out set. We reproduce it on a public dataset: we train nnU-Net 3D, extract a few summary numbers per patient, build the intervals, and check whether they contain the true ratio as often as promised.

The pipeline in 9 steps

#	Where	What happens	What comes out
1	Both	Set up tools: conda env, GitHub repo, Kaggle account	Working environments
2	Laptop	Write the CARE code (care/ package) and test it on synthetic data	Tested code, pushed to GitHub
3	Kaggle	Clone repo, download MSD Task01, preprocess, run a 5-epoch smoke test	Preprocessed data, epoch time, tiny stats file
4	Kaggle	Train nnU-Net 3D on one fold (long step)	Trained checkpoint
5	Laptop	While step 4 runs: re-read the paper, write the analysis notebook on fake statistics	Notebook 2 ready
6	Kaggle	Predict the held-out patients and save per-patient summary numbers	stats.npz (a few MB)
7	Laptop	Download stats.npz and check it	data/stats.npz
8	Laptop	Run CARE and the baselines on 100 random calibration/test splits	Coverage and width tables
9	Laptop	Figures, write-up, presentation	A note and slides for the professor

The one-sentence logic of the split. Everything that needs a GPU or huge files (training, prediction on 3D volumes) happens on Kaggle and is compressed into one tiny file; everything that is thinking, code and small numbers happens on the laptop.

Part 2. The paper explained

2.1 Setting and notation

A scan has *n* voxels (3D pixels). For each voxel *i* there are two ground-truth maps, y_A and y_B, equal to 1 if the voxel belongs to region A (the target, e.g. necrosis) or region B (the reference, e.g. the whole tumour), else 0. The network outputs, per voxel, probabilities g_A and g_B between 0 and 1.

- **True ratio:** r = (sum of y_A) / (sum of y_B).
- **Predicted ratio:** r-hat = (sum of g_A) / (sum of g_B). It uses the soft probabilities, not thresholded masks.
- **Example (NTR):** A = necrotic / non-enhancing core, B = whole tumour (edema + necrosis + enhancing) [VERIFY the exact label definitions in the paper].

2.2 The goal

Produce an interval [lo, hi] so that the true ratio lies inside with probability at least a chosen level (the paper uses 0.68). **Coverage** is the observed fraction of test patients whose true ratio is inside their interval. If the level is 0.68 and coverage is 0.87, the method is valid but conservative. If coverage is 0.30, the intervals are too optimistic. A flagged case is one whose interval crosses a clinical threshold (their example: 0.25), meaning a human should look.

2.3 Two sources of uncertainty

(a) Estimation uncertainty: a ratio of two sums is noisy. Treat each voxel as a sample, with x = g_B and y = g_A. The ratio of the two means has an approximate variance given by the *delta method* (a first-order Taylor expansion):

```
SE  =  (1/n) * [ var(y)/mu_x^2  +  var(x)*mu_y^2/mu_x^4  -  2*cov(x,y)*mu_y/mu_x^3 ]
half-width  beta = sqrt(SE) / sqrt(alpha)       (Markov inequality, probability >= 1 - alpha)
```

[VERIFY] The summary I read wrote the first term as var(y)/mu_x instead of var(y)/mu_x^2. The standard delta method gives mu_x^2 (units must match). Check the paper's Equation 6/7 and note which one it prints.

(b) Calibration uncertainty: the network's probabilities are biased. A network is *calibrated* if, among voxels where it says 0.7, about 70% really are positive. Two measures:

- **Volume bias (V-Bias)** = mean over voxels of (g - y): the signed error on the predicted volume of a region, per patient.
- **Calibration error (CE, estimated by ECE)** = mean of |g - P(y=1 | g)|. A known inequality gives |V-Bias| ≤ CE, so ECE is a more conservative stand-in for V-Bias.

On a held-out **calibration set** of patients, compute |V-Bias| (or ECE) for A and for B, and take the (1 - delta/2) quantile: q_A, q_B. Then, for a new patient with mean predictions mean(g_A) and mean(g_B), the ratio of *expected* labels lies between:

```
lower = (mean_gA - qA) / (mean_gB + qB)          upper = (mean_gA + qA) / (mean_gB - qB)
```

because shrinking the numerator and growing the denominator gives the smallest possible ratio, and the opposite gives the largest.

2.4 Putting them together

A union bound over the failure events gives the final interval: [r-hat - $\text{eps}_{\text{lower}}$ - beta, r-hat + $\text{eps}_{\text{upper}}$ + beta] with probability at least 1 - alpha - delta. With the target 0.68, alpha + delta = 0.32, and the split is chosen by grid search to give the smallest average width on the calibration set. Two versions: **CARE (V-Bias)** is tighter; **CARE (ECE)** is more conservative.

2.5 Baselines in the paper (and why each one fails)

Baseline	What it does	Why it falls short (paper's finding)
Subsampling / bootstrap of voxels	Resample voxels, recompute the ratio, take percentiles	Captures only random noise and treats the network as truth; coverage about 5-9%

Baseline	What it does	Why it falls short (paper's finding)
Deep ensembles, MC dropout	Disagreement between several predictions	Under-covers (27-46%); no appropriate prior
Split conformal prediction	Interval = r-hat ± q, q = 0.68 quantile of |r - r-hat| on the calibration set	Valid on average (about 66-71%) but the width is the same for every patient: it cannot adapt to small tumours

2.6 Results to aim for (nnU-Net 3D, MSD Task01, NTR, level 0.68)

Method	Coverage in the paper (approx.)
Subsampling / bootstrap	about 9% / 8%
Conformal prediction	about 67%
CARE (V-Bias)	about 87%
CARE (ECE)	about 94%

- **Adaptiveness:** CARE intervals are wider for small tumours and narrower for large ones; conformal intervals are nearly constant.
- **Decomposition:** most of the width comes from the calibration term, not from the estimation term. This is their main message: *miscalibration is the dominant source of uncertainty*.
- **Limits stated by the authors:** calibration and test data must share a distribution (scanner or population shift breaks the guarantee); tightness depends on how well calibrated the network is; only public data tested. The debiased estimator (their Appendix B.3) is derived but not evaluated: a possible extension.

Part 3. Scope: what we do and what we skip

Included (core reproduction)	Skipped (too costly for a first reproduction)
MSD Task01 BrainTumour, ratio NTR	BraTS21 and KiTS23 datasets
nnU-Net 3D, one fold, reduced epoch count	nnU-Net 2D, nnFormer, UNETR++
CARE (V-Bias) and CARE (ECE)	Nested 5-fold cross-validation
Conformal prediction baseline; voxel bootstrap baseline	Deep ensembles, MC dropout
Coverage at 0.68; width versus tumour size; width decomposition	KDE-based ECE, temperature scaling study

Deviations we will have (write them in the log, Part 8)

- Training length is shorter than the original schedule, which can change calibration and hence all coverage numbers.
- Calibration and test sets are random halves of the held-out patients of one fold, repeated 100 times, instead of 10% of the training data in nested folds.
- Training details (optimizer, learning rate, batch size, patch size) are nnU-Net defaults because the paper text we read does not specify them.

How to judge success. Matching the paper to within a few points of coverage is good. A *different* number that you can explain (shorter training, different split, different voxel set) is also a valid scientific result. A number you cannot explain is the only bad outcome.

Why a partial reproduction is worth discussing with a professor

- It shows you understood the method end to end, not only the abstract.
- The list of gaps (missing hyper-parameters, the formula to verify, the unevaluated debiased estimator) is itself useful material for a conversation.
- It leads to concrete research questions, listed in Part 9.

Part 4. Where each task runs, and why

Decision rule. If it needs a GPU or large files, it runs on Kaggle. If it is thinking, code, or small numbers, it runs on the laptop in VS Code.

	Laptop (VS Code)	Kaggle (notebook)
Machine	Intel i5-1135G7, 8 GB RAM, MX330 with 2 GB (too small and too old for nnU-Net 3D)	Free GPU (P100 or T4) and more RAM [VERIFY current offer]
Strengths	Permanent files, git, fast editing, no time limits, debugging with tests	A GPU for hours at no cost; large temporary disk
Weaknesses	Cannot train; inference on 3D volumes is slow and risks running out of memory	Sessions are temporary and capped; weekly GPU quota; awkward for debugging
We do there	Write care/ code; synthetic tests; notebook 2 (analysis); figures; write-up; presentation	Install nnU-Net; download and preprocess data; train; predict; extract statistics
Files that live there	Code (git), stats.npz, results	MRI data (about 7 GB), preprocessed data, checkpoints

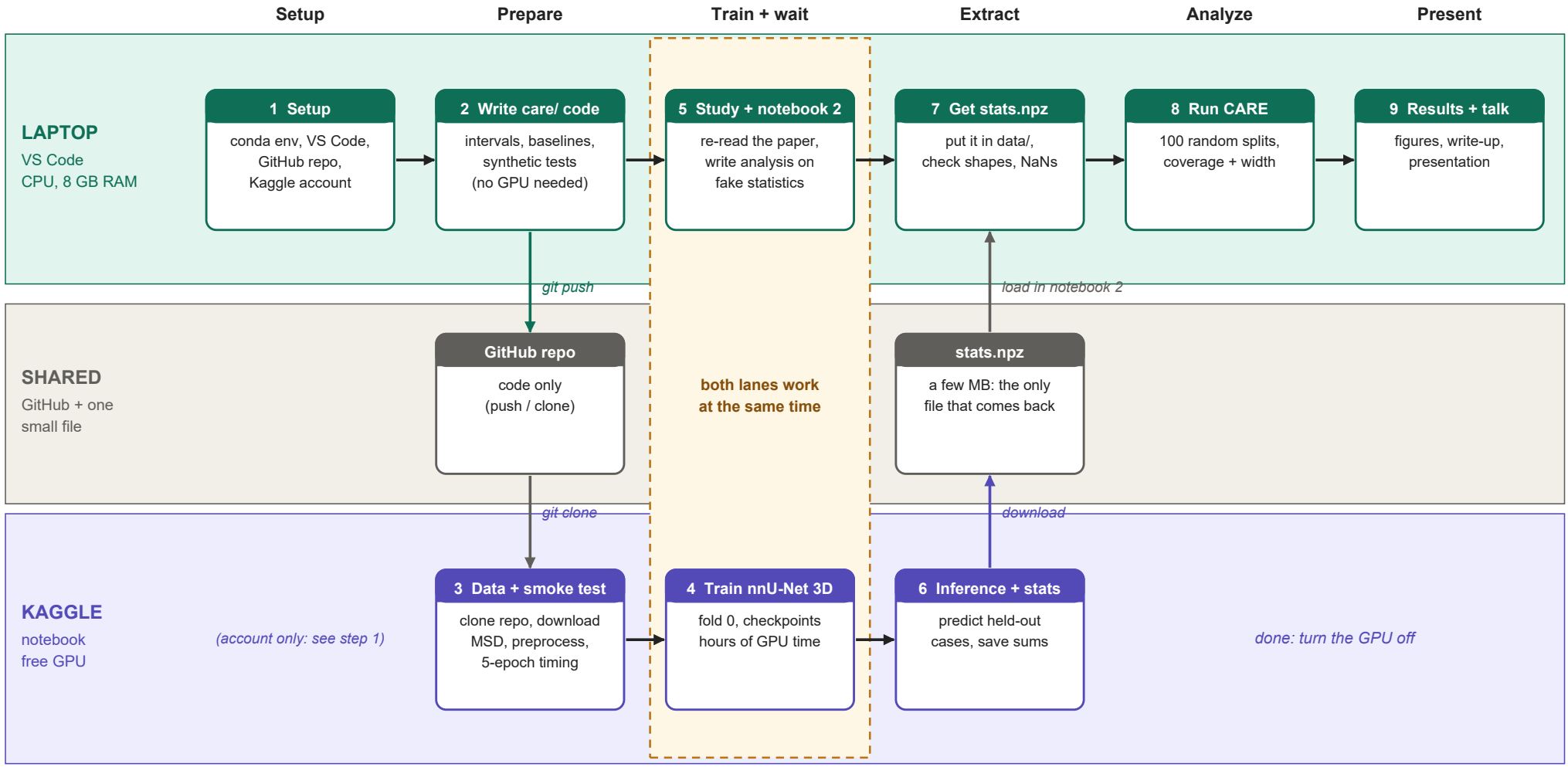
How the two sides communicate

They never talk directly. They exchange **files**, through two channels:

What	Direction	Channel	How
Code (care/, scripts)	Laptop → Kaggle	GitHub	git push on the laptop; git clone (or git pull) in the first Kaggle cell. A private repo needs a token in Kaggle Secrets [VERIFY]
Result (stats.npz, a few MB)	Kaggle → Laptop	File download	Download from the notebook Output tab (or a Kaggle Dataset, or commit the small file to GitHub)

- **Never** move MRI volumes or checkpoints through GitHub. They are large, and you do not need them on the laptop.
- **Why not do everything on Kaggle?** Sessions end; your code would be scattered across notebook versions; debugging is slow. Keeping the logic in a git repository on the laptop makes the work reproducible.
- **Why not do everything on the laptop?** It cannot train the network, and prediction on full 3D volumes strains 8 GB of RAM.
- **Why the small file is enough.** CARE needs only a handful of sums per patient (Part 7). Compressing each 3D volume into those sums is what lets the analysis run on a modest laptop.

Part 5. The pipeline schema



How to read: time runs left to right, numbers give the order of work, each lane is a place.

Laptop (VS Code) · Shared channel · Kaggle (GPU) · Parallel phase · Order of work / file transfer

Reading the schema

- **Three lanes:** teal = your laptop in VS Code; gray = the shared channel between them; purple = the Kaggle notebook.
- **Six columns = time.** Setup, prepare, train and wait, extract, analyze, present.
- **Numbers 1-9** give the order of work. Steps 4 and 5 are the parallel phase (amber dashed band): Kaggle trains while you study and write the analysis.
- **Two vertical hand-offs.** Left side (column 2): code goes laptop → GitHub → Kaggle. Right side (column 4): the small stats file goes Kaggle → laptop. These are the only two crossings between the lanes.
- **After step 6** the GPU is no longer needed; everything else is on your laptop.

The pipeline as a data story

Stage	Object	Size	Lives
Raw data	484 labelled brain MRI volumes (4 modalities)	about 7 GB	Kaggle
After preprocessing	Resampled, normalized volumes	several GB	Kaggle
After training	Network weights (checkpoint)	hundreds of MB	Kaggle
After prediction	Per-patient probability maps g_A, g_B	about 100 MB each (never stored)	Kaggle, in memory
After extraction	A dozen numbers per patient	a few MB in total	Travels to the laptop
After CARE	Intervals, coverage, widths, figures	KB	Laptop

Each arrow in the pipeline shrinks the data. Training and prediction need the big objects, so they stay on the machine that has them; the analysis needs only the smallest object, so it can run anywhere.

Part 6. Step-by-step instructions

STEP 1	Set up the tools	LAPTOP + KAGGLE

Why here. Both places must be ready before any work is lost to setup problems, and the repository is how code travels between them.

How.

- **Laptop:** the conda environment `care` (Python 3.12, CPU PyTorch, NumPy, SciPy, pandas, matplotlib, nibabel, scikit-learn) is already installed. In VS Code install the Python and Jupyter extensions and select the `care` interpreter.
- **Laptop:** create the project folder, initialize git, create an empty GitHub repository, and push a skeleton (README and the folder layout of Part 8).
- **Kaggle:** create an account, verify your phone number (needed to enable GPU and Internet [VERIFY]), create a notebook, set Accelerator = GPU and Internet = On.

```
conda activate care
cd C:\Users\<you>\care-reproduction
git init
git add .
git commit -m "skeleton"
git remote add origin https://github.com/<you>/care-reproduction.git
git push -u origin main
```

Output. A working interpreter on the laptop; a GitHub repository; a Kaggle notebook with GPU and Internet enabled.

Be able to explain. Why there are two separate environments, and why the repository is the bridge between them.

STEP 2	Write the CARE code and test it on synthetic data	LAPTOP - VS Code

Why here. This is pure NumPy on small arrays: no GPU needed, instant feedback, and you can test against *known truth* because you invent the data.

How.

- Create the package `care/` with three modules: `intervals.py` (the SE term, the CARE interval, the alpha/delta grid search), `baselines.py` (conformal, voxel bootstrap), `metrics.py` (coverage, mean width, width versus tumour size).
- Write a generator of fake per-patient statistics: simulate voxel probabilities for about 100 fake patients with a controlled miscalibration (for example shifting the logits by a fixed bias) and tumours of varying size; compute the same sums as the real extraction (Part 7).
- Write tests that assert what the theory predicts: (i) conformal coverage is close to 0.68; (ii) CARE coverage is at least 0.68; (iii) intervals are wider for small tumours; (iv) with zero bias and large n the interval shrinks; (v) the denominator stays positive.
- Push to GitHub when the tests pass.

```
python -m pytest tests/ -q
git add care tests
git commit -m "CARE intervals, baselines, synthetic tests"
git push
```

Output. A tested package on GitHub, and a fake `stats.npz` with the same structure as the real one.

Be able to explain. What each formula computes, what each test proves, and why testing on synthetic data with a known answer comes before touching real data.

STEP 3	Download data, preprocess, run a smoke test	KAGGLE - GPU notebook

Why here. The dataset is about 7 GB and preprocessing resamples every volume; both need Kaggle's disk and CPU. The smoke test uses the GPU, so it must run where the GPU is.

How.

- Cell 1: install and clone. Cell 2: set the nnU-Net paths on the large temporary disk (`/kaggle/temp` [VERIFY]). Cell 3: download MSD Task01 from the official source and extract it (check the licence; confirm the download with your professor or the project rules first).
- Convert the dataset to nnU-Net's format, then run planning and preprocessing. Preprocessing resamples all scans to a common voxel spacing, normalizes each modality, and crops to the non-zero region.
- **Smoke test:** train for only a few epochs (a trainer such as `nnUNetTrainer_5epochs`; list available trainers in the installed package [VERIFY]) and extract statistics for about 5 patients. The numbers are meaningless; you are checking that nothing crashes, measuring the time per epoch, and checking the file format.

```
!pip install nnunetv2
!git clone https://github.com/<you>/care-reproduction.git
import os
os.environ['nnUNet_raw']='/kaggle/temp/nnUNet_raw'
os.environ['nnUNet_preprocessed']='/kaggle/temp/nnUNet_preprocessed'
os.environ['nnUNet_results']='/kaggle/working/nnUNet_results'
!nnUNetv2_convert_MSD_dataset -i /kaggle/temp/Task01_BrainTumour
!nnUNetv2_plan_and_preprocess -d 1 --verify_dataset_integrity
!nnUNetv2_train 1 3d_fullres 0 -tr nnUNetTrainer_5epochs
```

Output. Preprocessed data; the measured time per epoch; a tiny `stats.npz` that notebook 2 can already load.

Be able to explain. What preprocessing does, why we run a smoke test, and how the epoch time decides the plan (full training here or on a rented GPU).

STEP 4 — Train nnU-Net 3D on one fold — KAGGLE - GPU notebook

Why here. This is the only step that needs hours of GPU, and it does not depend on the CARE maths (the paper's loss, cross-entropy plus soft Dice, is nnU-Net's default).

How.

- Fold 0 trains on about 80% of the 484 labelled patients (roughly 387) and keeps about 97 as held-out patients. Those held-out patients become the calibration and test pool for CARE: the network never saw them.
- Choose the epoch count from the smoke-test timing (for example 250) and **write it in the deviation log**. nnU-Net defines an epoch as a fixed number of iterations, so epochs scale linearly with time.
- Sessions are temporary: copy the results folder to a place that survives (the notebook output or a Kaggle Dataset), and resume interrupted runs with the continue flag. If the run cannot fit in the session limits [VERIFY], options are fewer epochs, several resumed sessions, or a rented GPU.

```
!nnUNetv2_train 1 3d_fullres 0 -tr nnUNetTrainer_250epochs
# if interrupted:
!nnUNetv2_train 1 3d_fullres 0 -tr nnUNetTrainer_250epochs --c
```

Output. A trained model (checkpoint) for fold 0, plus the training log.

Be able to explain. What an epoch is, why we hold patients out, and how training length can influence calibration (and therefore the CARE intervals).

STEP 5 — Study the paper and write notebook 2 (in parallel with step 4) — LAPTOP - VS Code

Why here. Waiting time on the GPU is study time on the laptop. Notebook 2 only needs the file format, so you can finish it on the fake statistics from step 2.

How.

- Re-read Part 2 and then the paper's Sections 3 and 4; tick the checklist in Appendix A. Practise explaining each formula on paper without notes.
- Notebook 2 cells: load `stats.npz`; sanity checks; loop 100 times over random 50/50 calibration/test splits of the held-out patients; in each loop fit CARE (V-Bias and ECE) and the conformal and bootstrap baselines on the calibration half and evaluate coverage and width on the test half; aggregate to mean ± standard deviation;

produce the coverage table, the width-versus-tumour-size plot and the width decomposition.

- Keep the logic in `care/`; the notebook should only call functions and show results.

Output. Notebook 2 running end to end on fake statistics.

Be able to explain. Each cell in the notebook and why the splits are repeated (a single small test set gives a noisy coverage).

STEP 6 Predict the held-out patients and extract statistics — KAGGLE - GPU notebook

Why here. Prediction needs the trained model, the GPU and the 3D volumes, none of which can leave Kaggle. So we compress each prediction into a few sums on the spot instead of storing probability maps (tens of GB).

How.

- Load the model with nnU-Net's predictor in the Python API, using fold 0 and the final checkpoint. Take the list of held-out patients from the fold split file.
- For each patient: predict softmax probabilities; set g_A = probability of the necrosis/non-enhancing class and g_B = 1 minus the background probability [VERIFY label mapping: 1 edema, 2 non-enhancing/necrotic, 3 enhancing]; load the ground-truth labels for y_A, y_B; compute the statistics of Part 7.
- Decide and **record** whether mirroring test-time augmentation is on (it makes prediction several times slower), and which voxels count (whole volume or a region), see Appendix B.
- Save everything with `numpy.savez_compressed('stats.npz', ...)` and run sanity checks in the notebook: about 97 patients, no NaN, predicted versus true ratio scatter looks sensible.

Output. `stats.npz` plus a short log of all settings (epochs, fold, TTA, voxel set).

Be able to explain. Why we save sums and not images, and what each saved number is used for.

STEP 7 Bring the statistics to the laptop — LAPTOP - VS Code

Why here. From here on nothing needs a GPU, and your laptop becomes the only working place.

How.

- Download `stats.npz` from the Kaggle notebook output (or commit it to GitHub, it is only a few MB) and put it in `data/`.
- Check: keys, shapes, number of patients, no NaN or infinite values, the true ratio between 0 and 1, and that the training settings in the log match what you will report.

```
import numpy as np
d = np.load('data/stats.npz')
print({k: d[k].shape for k in d.files})
```

Output. `data/stats.npz` verified.

Be able to explain. That this small file is a faithful summary of the heavy computation done on Kaggle.

STEP 8 Run CARE and the baselines — LAPTOP - VS Code

Why here. It is light CPU work (seconds per split), so it runs comfortably on 8 GB of RAM, and you can re-run it as often as you like while you explore.

How.

- Run notebook 2 on the real statistics. Compare coverage at level 0.68 with the paper's Table 1 values (Part 2.6).
- Explore, one change at a time: the confidence level (for example 0.5 to 0.95, the paper's *tunability*), the number of ECE bins, the voxel set, with or without the alpha/delta grid search. Note effects in the log.
- If coverage is below the target, check the formula you flagged in Part 2.3, the label definitions, and the denominators close to zero; do not tune anything to match the paper's numbers.

Output. Tables of coverage and width (mean ± standard deviation over 100 splits) for conformal, bootstrap, CARE (V-Bias), CARE (ECE).

Be able to explain. Each number in the table, what it says about validity (coverage) and usefulness (width), and the reasons for any gap with the paper.

STEP 9 Figures, write-up, presentation — LAPTOP - VS Code

Why here. This is where understanding becomes something another person can follow.

How.

- Figures: coverage bars with the 0.68 line; width versus tumour size (adaptiveness); stacked width decomposition (calibration versus estimation).
- A one-to-two page note: goal, method in your own words, what you did, results next to the paper's, deviations, open questions. A README that lets someone re-run everything.
- Slides using the five-minute story and the questions of Part 9. State clearly which parts were AI-assisted and that you checked the code and numbers yourself.

Output. A note, slides and a clean repository to send to the professor.

Be able to explain. The full pipeline, from the clinical motivation to the limits, without notes.

Part 7. Inside stats.npz and how CARE uses it

For every held-out patient the extraction script stores a row of sums. These are called *sufficient statistics* here because CARE can be computed from them alone.

Field	Meaning	Used for
case_id	Patient identifier	Traceability
n	Number of voxels counted	All averages
sgA, sgB	Sums of g_A and g_B over voxels	r-hat = sgA / sgB; means
sgA2, sgB2, sgAgB	Sums of g_A^2, g_B^2, $g_A \cdot g_B$	Variance and covariance in the SE term
syA, syB	Sums of y_A and y_B (ground truth)	True ratio r = syA / syB; V-Bias
binsA, binsB	For 15 probability bins: voxel count, sum of probabilities, sum of labels	ECE for A and B, per patient
tumour_size	syB in voxels	Adaptiveness analysis

From the fields to the interval (what notebook 2 does)

- **Per patient:** r-hat = sgA / sgB; true ratio r = syA / syB; V-Bias_A = (sgA - syA) / n and V-Bias_B likewise; ECE_A from the bins.
- **Estimation term:** means mu = sums / n; variances and covariance from the second-order sums; SE from the delta-method formula; beta = sqrt(SE) / sqrt(alpha).
- **Calibration step (calibration half only):** quantiles q_A, q_B of |V-Bias| (or ECE) at level 1 - delta/2; grid search over alpha with delta = 0.32 - alpha (for level 0.68).
- **Test half:** build the interval for each patient with those quantiles; coverage = fraction of patients with lower ≤ r ≤ upper; width = upper - lower.
- **Conformal baseline:** q = 0.68 quantile of |r - r-hat| on the calibration half; interval r-hat ± q.
- **Voxel bootstrap baseline** needs voxel-level information; the stored sums are enough for a parametric version (resampling voxels with the same mean and variance), which we will note as a deviation.

Open point. The paper's V-Bias is a mean over voxels. Whether it counts the whole image or only a region around the brain changes the numbers a lot, since most voxels are background. Decide, record, and test the sensitivity (Appendix B).

Repository layout

```
care-reproduction/
  README.md
  environment.yml
  care/                    # shared by both notebooks
    intervals.py           # SE term, CARE bounds, alpha/delta grid
    baselines.py           # conformal, bootstrap
    metrics.py             # coverage, width, adaptiveness
  tests/
    test_synthetic.py      # laptop-only checks
  notebooks/
    01_kaggle_train_extract.ipynb   # runs on Kaggle
    02_laptop_analysis.ipynb        # runs in VS Code
  scripts/
    extract_stats.py       # called by notebook 1
  data/
    stats.npz              # downloaded from Kaggle
  results/                 # tables and figures
```

Part 8. Time, cost, risks, deviation log

Time budget (estimates, to be replaced by your measurements)

Phase	Hours	Where	Why it takes time
Reading the paper closely	3-5	Laptop	The maths must be understood, not skimmed
Setup, accounts, repository	2-3	Both	Installs, tokens, settings
Writing care/ and synthetic tests	6-10	Laptop	Debugging formulas; edge cases such as tiny tumours
Data download and preprocessing	1-2	Kaggle	7 GB to download, every volume resampled
Smoke test	1-2	Kaggle	Finds the problems cheaply
Training	6-25 (GPU-dependent)	Kaggle	3D convolutions on large patches, fixed iterations per epoch
Prediction and statistics	0.5-1.5	Kaggle	Sliding-window prediction on each held-out volume
Analysis and figures	4-8	Laptop	Mostly thinking and checking
Write-up and slides	4-6	Laptop	Explaining clearly takes longer than computing
Total	about 30-60 working hours		Spread over 2-4 weeks; the training wait overlaps with study

Cost. Free on Kaggle if training fits the quota. Otherwise a rented GPU (an RTX 4090 at roughly $0.3-0.7 per hour in mid-2026 listings [VERIFY on the provider's page]) costs about $5-15 for the whole project.

Risks and how to handle them

Risk	Signal	Response
Training does not fit in free sessions	Smoke-test epoch time x epochs exceeds the limits	Fewer epochs, resume across sessions, or rent a GPU for a few hours
Out-of-memory on Kaggle	Kernel dies in prediction	Predict one case at a time, free memory, disable mirroring
Formula error	Coverage far below target on synthetic data	Re-derive; check the paper's equation; unit tests
Intervals blow up	Negative or tiny denominator	Clip the denominator, flag the case, report how many
Coverage far from the paper	Table 1 mismatch	Check label definitions, voxel set, training length; document, do not tune
Lost work	Session expires	Checkpoints and stats saved outside the session; code only in git

Deviation log (fill in as you go)

Item	Paper	Our reproduction	Why / effect
Epochs	[VERIFY]		
Folds / splits	Nested 5-fold, 10% calibration	Fold 0, random halves x100	Compute
Architectures	4	nnU-Net 3D only	Compute
Voxel set	[VERIFY]		
Test-time augmentation	[VERIFY]		
ECE bins	[VERIFY]	15	

Part 9. Presentation kit

The five-minute story (six beats)

- **1. Problem.** AI segmentation gives a biomarker ratio with no sign of how reliable it is; near a clinical threshold this matters.
- **2. Gap.** Resampling and ensembles ignore that the network is miscalibrated; conformal prediction is valid but cannot adapt to the case.
- **3. Idea.** Combine the noise of a ratio estimator with the calibration error measured on held-out patients, in a post-hoc, training-free interval.
- **4. What I did.** Trained nnU-Net 3D on MSD Task01; extracted per-patient sums on a GPU; built CARE intervals and baselines on my laptop; repeated over 100 calibration/test splits.
- **5. Results.** Coverage and width next to the paper's numbers; adaptiveness; what dominated the width; honest list of deviations.
- **6. Limits and next steps.** Distribution shift, conservatism, the unevaluated debiased estimator; questions for the professor.

Questions you may be asked, with short answers

Question	Answer
Why not just bootstrap the voxels?	It only captures the random noise of the ratio and treats the network's probabilities as truth. In the paper it covers 5-9% when 68% is promised.
Why not use conformal prediction alone?	It is valid on average, but its interval has the same width for every patient, so it cannot be tighter for easy cases or wider for small tumours.
Why is CARE conservative (coverage above 68%)?	It uses union bounds, worst-case quantiles and a loose Markov bound. That buys validity at the cost of width.
What does coverage mean?	The fraction of test patients whose true ratio lies inside their interval, to be compared with the promised level.
What if the test hospital is different?	The guarantee assumes calibration and test data come from the same distribution; the authors list this as a limitation.
Why one fold and fewer epochs?	Compute budget. Training length affects calibration, so I document it and report the effect as a deviation.
Did you write the code yourself?	It was AI-assisted. I reviewed it, tested it on synthetic data with known answers, and can explain every function and formula.
What would you do next?	Test the debiased estimator; test calibration under a scanner shift; try another ratio biomarker.

Ideas to propose to the professor

- Are the intervals narrow enough to be clinically useful against a threshold such as 0.25?
- Does a better-calibrated network (calibration-aware loss, recalibration) shrink the intervals, and by how much?
- How big is the bias of the plain ratio estimator at realistic tumour sizes, and does the paper's debiased estimator fix it?
- How do the guarantees degrade under distribution shift, and can a label-free calibration estimate help?

Glossary

Term	Meaning
Segmentation	Labelling each voxel of a scan as belonging to a region (tumour, necrosis, background)
Softmax probability	The network's per-voxel probability for each class
Calibration	Agreement between predicted probabilities and observed frequencies
V-Bias	Mean of (prediction - truth) over voxels: the signed volume error

Term	Meaning
ECE	Expected calibration error: gap between confidence and accuracy, averaged over probability bins
Coverage	Fraction of cases whose true value lies in the interval
Conformal prediction	Interval built from the empirical error quantile on a calibration set
Delta method	First-order Taylor approximation of the variance of a function of averages
Markov inequality	$P(\|error\| \geq k \cdot SD) \leq 1/k^2$ type bound, loose but assumption-light
Union bound	$P(A \text{ or } B) \leq P(A) + P(B)$; lets us add two error budgets
Epoch	One fixed batch of training iterations (nnU-Net: a fixed number per epoch)
Checkpoint	Saved network weights
Fold	One of the splits of the data into training and held-out patients
TTA	Test-time augmentation: predicting on mirrored copies and averaging
Sufficient statistics	The few sums from which an analysis can be computed without the raw data

Appendix A. Facts to verify against the PDF

Tick each item once checked in the paper's PDF (link on the PMLR page) and write the value in the deviation log.

Done	Item to verify
	The variance formula: is the first term var(y)/mu_x or var(y)/mu_x^2 (delta method gives the latter)?
	Exact label definitions of regions A and B for NTR on MSD Task01
	Which voxels define n and the V-Bias average (whole volume or a region)
	Number of ECE bins, and how ECE is computed per patient
	Training schedule and any hyper-parameters stated in the appendix
	Whether test-time augmentation was used at prediction time
	Exact quantile definitions q (level 1 - delta/2) and the grid used for alpha/delta
	Exact numbers in Table 1 for nnU-Net 3D (the values in Part 2.6 are rounded from a summary)
	Whether the authors have released code (none was found when this guide was written)

Appendix B. Decisions still open

Decision	Options	Default we propose
Voxel set	Whole volume / bounding box of the brain / region around the tumour	Decide after reading the paper; test the sensitivity
Number of epochs	100, 250, 500	250 if it fits; else 100, documented
Calibration/test split	50/50 of ~97 patients, repeated 100 times	50/50 x 100
Test-time augmentation	On / off	Off if memory or time is a problem; documented
Bootstrap baseline	Exact voxel resampling / parametric from sums	Parametric from sums; documented
Platform for training	Free Kaggle / rented GPU	Decide after the smoke test

Rule for the whole project. Never present a number you did not compute yourself, and never tune a setting to reproduce a published value. Report what you did, what you got, and why it may differ.